

09 Source Analysis

The goal of this assignment is to perform source analysis of the Insecure Bank project, and to answer some questions about source analysis and the results you have generated.

Build Insecure Bank

The project you're going to examine is Insecure Bank, which is a deliberately broken bank web site.

The first step is to build Insecure Bank, as SonarQube will integrate with the build to do its analysis.

Get the source code like this (do `sudo apt-get install -y git` first if necessary):

```
git clone https://github.com/jknudsen-synopsys/insecure-bank
```

You'll need a Java environment and the Maven build tool. You can get them like this:

```
sudo apt-get update
sudo apt-get install -y openjdk-11-jdk maven
```

Now you should be ready to build. Go to the *insecure-bank* directory and try this:

```
mvn package
```

After a minute or two, you should see a **BUILD SUCCESS** message.

Log In to SonarQube

Log in to your SonarQube account. I'm running SonarQube on my Duke VM, which is here:

<http://vcm-18637.vm.duke.edu:9000/>

Your login is your first name (tony, annie, priya, kyle, and cady) and your password is code1234. Change it if you wish.

You will need an API token. In the top right of SonarQube, click on your user name and choose **My Account**, then click on the **Security** tab. Generate a new token and copy its value. You'll need this value in the next step.

Analyze Insecure Bank with SonarScanner for Maven

Because Insecure Bank uses the `mvn` build system, use the following:

<https://docs.sonarqube.org/latest/analysis/scan/sonarscanner-for-maven/>

You don't have to download or install anything, but you do need to tell Maven about the SonarQube scanner.

Follow the instructions on the page. The *settings.xml* file that you need to edit is located at `/usr/share/maven/conf/settings.xml`. You will need to use `sudo` to edit the file. Add the `pluginGroup` and the `profile` as described in the web page, then adjust the `sonar.host.url` in the profile so that it points to my SonarQube instance.

This part is a little tricky, but I did it in the class on March 17 so you can refer to the recording for that.

Attempt to analyze the source code using the token you generated in the previous step (replacing `myAuthenticationToken`). Also, make sure you include a project key and a project name that has your user name in it (replacing `myName` below). If you leave this out, your analysis will go to my existing project and you won't be able to find it. (You will be able to see everyone else's analyses.)

```
mvn clean verify sonar:sonar -Dsonar.login=myAuthenticationToken
-Dsonar.projectKey=myName:insecure-bank -Dsonar.projectName="myName Insecure
Bank"
```

If you get an error, you probably have a problem in *settings.xml* you need to fix.

If it works, it will take a few minutes until the analysis is done, then a little more time before you can see the results in SonarQube in your browser.

Analyze Insecure Bank with semgrep

Check to see if Python and pip are installed:

```
python3 --version  
pip3 --version
```

Install Python and pip if necessary:

```
sudo apt-get install -y python3  
sudo apt-get install -y python3-pip
```

Now install semgrep:

```
pip3 install semgrep
```

Navigate to the Insecure Bank directory. Analyze the code using Java checkers:

```
semgrep --config=p/java
```

Browse through the available rulesets:

<https://semgrep.dev/rulesets>

Choose at least one other ruleset and analyze Insecure Bank again. To answer the questions below, choose one ruleset and use those results.

Questions

Leave your analysis in SonarQube so I can see it there.

Answer the following questions.

1. SonarQube reports on three types of issues. Describe each one in your own words. Is it useful to have issues divided up this way?
2. How many issues did SonarQube report? How many did semgrep report?
3. Does it make sense to separate **Issues** from **Security Hotspots**?

4. Examine **Security Hotspots**. Choose three different hotspots with different types. For each one, click **Get Permalink** and submit the link. Would you follow SonarQube's advice? Would you take some other action? Is the issue a false positive?
5. Choose three issues from the semgrep output. For each one, submit the semgrep output. Are you able to find the same issue in SonarQube? (Try to find issues that are in both tools. I'm pretty sure both report on SQL injection, and I suspect there are others.) Which tool helped you understand the problem better?
6. If you were part of a development team, which tool would you rather use? Why?
7. Assume you are part of the Insecure Bank team and it is a real project for a real bank. Write a policy (no more than one page) for how you want the development team to use static analysis. Cover automation, how often static analysis is run, how results are handled, what results are acceptable for deploying the project, and anything else that seems relevant.